Metal-Sci: A Scientific Compute Benchmark for Evolutionary LLM Kernel Search on Apple Silicon

Anonymous Authors
Anonymous Affiliation
anonymous@example.org

Abstract

LLM-driven code search has produced impressive results on GPU ML kernels (GEMM, attention, convolution), but the optimization patterns in scientific computing are systematically different: bandwidth-bound stencils with halo tiling, register-blocked all-pairs interactions, multi-field Boltzmann updates, neighbour-list molecular dynamics with atomics, parallel-chain MCMC, multi-kernel nonlinear PDE solvers, and butterfly/data-shuffle spectral transforms. We present METAL-SCI, a 10-task benchmark of scientific Apple-Silicon Metal compute kernels spanning six optimization regimes. Every task ships a CPU reference, a roofline ceiling, and a naive but correct seed; the fitness signal is the geometric mean of *achieved/ceiling* across three problem sizes, with any tolerance failure forcing the score to zero. We release the harness and seeds and report a uniform 10-iteration Claude Opus 4.7 sweep over all ten tasks on Apple M1 Pro. In-distribution speedups span $1.00\times$ to $10.6\times$. *The held-out evaluation is sharper than the in-distribution one and exposes overfitting that the geometric mean hides.* On HMC, the `template <uint D>` dispatch that took $d{=}8$ from 121 to 970 GFLOPS hard-codes branches for $d{\in}\{8, 16, 32\}$ and *returns wrong answers* at the held-out $d{=}24$. On LBM, a $32{\times}2$ threadgroup geometry tuned for the training sizes regresses to $0.64\times$ of the seed at 192^2. gradshaf, lj, fft3d, and nbody extrapolate cleanly. Metal-specific syntax (`[[max_total_threads_per_threadgroup]]` attribute placement, `half` as reserved fp16 keyword, MSL's lack of C++ lambdas) recurs as an out-of-distribution failure across iterations. The benchmark is designed for fast iteration: each candidate compiles, runs, verifies, and scores in seconds.

1 Introduction

LLM code-search systems — FunSearch [1], AlphaEvolve [2], and recent kernel-generation work [3] — evaluate language models as *optimisers* of executable artefacts. The choice of artefact matters: KernelBench targets PyTorch ML kernels (GEMM, attention, convolution) on CUDA, where canonical implementations are heavily represented in pretraining data and the optimisation surface is dominated by tiling and warp-level reductions.

Scientific computing exposes a different surface. Stencils are bandwidth-bound and reward halo and temporal blocking. All-pairs interactions are compute-bound and reward register blocking with shared-memory cooperative loads. Lattice Boltzmann ping-pongs nine distribution functions per cell with non-trivial pull-stream indexing. Cell-list molecular dynamics touches irregular memory through atomic counters. Hamiltonian Monte Carlo runs many chains in parallel where register pressure scales with the problem dimension d. Grad-Shafranov solvers chain a max-reduction with a variable-coefficient stencil. 3D FFTs are dominated by data-shuffle/butterfly patterns and twiddle-factor caching rather than tiling. Each pattern stresses a distinct dimension of the compiler/memory hierarchy, and canonical optimisations [5,6,7,8,11] differ regime-to-regime.

Preprint.

We target Apple Silicon's Metal compute pipeline. It is underrepresented in CUDA-centric training data — frontier models reliably mishandle Metal-specific syntax such as the `[[max_total_threads_per_threadgroup]]` kernel attribute, a real out-of-distribution generalisation test. Its unified memory model removes host-device copy plumbing, enabling sub-second compile-run-verify cycles per candidate that are essential for fast evolutionary loops. And it supports rich simdgroup intrinsics (`simd_max`, `simd_broadcast`) and threadgroup-memory tiling, giving the LLM a non-trivial optimisation surface.

Contributions. (i) A 10-task scientific compute benchmark over six optimisation regimes, each with a CPU reference, roofline-anchored fitness, and multi-size generalisation gate. (ii) A runtime-compiled harness that exposes compile errors, correctness diagnostics, and per-size GPU timing back to the LLM. (iii) Preliminary evolution runs across Claude Opus 4.7 and Gemini 3 on M1 Pro, qualitatively analysing which patterns each model recovers.

2 Benchmark tasks

Each task ships (a) a Metal source describing the problem, (b) a naive correct seed, (c) a CPU reference with task-specific tolerance, (d) three in-distribution sizes plus one held-out size, and (e) a roofline ceiling in either GFLOPS (compute-bound) or GB/s (bandwidth-bound). Tasks are grouped by optimisation regime.

2.1 Regular stencils (regime R1)

heat2d. Two-dimensional heat equation, 5-point stencil on $(N_x \times N_y)$ grid with Dirichlet boundaries. Per timestep, per interior cell:

$$u^{n+1}_{i,j} \;=\; u^n_{i,j} + \alpha\big(u^n_{i-1,j} + u^n_{i+1,j} + u^n_{i,j-1} + u^n_{i,j+1} - 4\,u^n_{i,j}\big)\,. \tag{1}$$

Bandwidth-bound at 8 B/cell; tests halo handling and temporal blocking.

wave3d. Three-dimensional acoustic wave equation, 7-point isotropic Laplacian, leapfrog in time:

$$u^{n+1}_{i,j,k} = 2u^n_{i,j,k} - u^{n-1}_{i,j,k} + \alpha\,\big(u^n_{i\pm1,j,k} + u^n_{i,j\pm1,k} + u^n_{i,j,k\pm1} - 6\,u^n_{i,j,k}\big), \tag{2}$$

where each $\pm$ expands to two terms. CFL stability requires $\alpha = (c\Delta t/\Delta x)^2 < 1/3$ (we use 0.18). 12 B/cell unique DRAM traffic. Tests register pressure, 2.5D blocking, and marching-axis choice.

2.2 Compute-bound (regime R2)

nbody. All-pairs gravitational N-body with leapfrog. Per body i per step:

$$\mathbf{a}_i \;=\; G\sum_j m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \varepsilon^2)^{3/2}}, \quad \mathbf{v} \mathrel{+}= \mathbf{a}\Delta t, \quad \mathbf{r} \mathrel{+}= \mathbf{v}\Delta t. \tag{3}$$

Self-interaction is masked by the softening ε. $\sim$20 FLOPs per pair; ceiling at peak FP32 GFLOPS. Tests register blocking and threadgroup cooperative loads.

hmc. Hamiltonian Monte Carlo on an anisotropic Gaussian target, $U(q) = \frac{1}{2}q^\top Aq$ with $A = \Sigma^{-1}$. One thread per chain; many chains in parallel. Each HMC step draws $p \sim \mathcal{N}(0, I)$ and runs L leapfrog inner steps with stepsize ε,

$$p \leftarrow p - \tfrac{\varepsilon}{2}Aq, \quad q \leftarrow q + \varepsilon p, \quad p \leftarrow p - \varepsilon Aq, \;\cdots, \quad p \leftarrow p - \tfrac{\varepsilon}{2}Aq, \tag{4}$$

followed by a Metropolis accept/reject with $\log u_{\text{acc}} < -\Delta H$. Correctness is verified statistically (sample mean and Frobenius covariance error vs. the target). Three sizes $(d, K) \in \{(8, 16\text{K}), (16, 4\text{K}), (32, 1\text{K})\}$ probe the register-pressure boundary; at d=32 per-thread state ($\sim$512 B) competes with the register file.

2.3 Multi-field, exotic memory (regime R3)

lbm. D2Q9 Lattice Boltzmann, fused pull-stream + BGK collision, periodic BC. With velocity table $\mathbf{c}_k$ and weights w_k, per cell per step:

$$f_k^{\mathrm{str}}(\mathbf{x}) = f_k^{\mathrm{in}}(\mathbf{x} - \mathbf{c}_k), \quad \rho = \textstyle\sum_k f_k^{\mathrm{str}}, \quad \mathbf{u} = \frac{1}{\rho}\sum_k \mathbf{c}_k\, f_k^{\mathrm{str}}, \tag{5}$$

$$f_k^{\mathrm{eq}} = w_k \rho\big(1 + 3(\mathbf{c}_k{\cdot}\mathbf{u}) + \tfrac{9}{2}(\mathbf{c}_k{\cdot}\mathbf{u})^2 - \tfrac{3}{2}\|\mathbf{u}\|^2\big)\,, \tag{6}$$

$$f_k^{\mathrm{out}} = f_k^{\mathrm{str}} - \tau^{-1}\big(f_k^{\mathrm{str}} - f_k^{\mathrm{eq}}\big)\,. \tag{7}$$

Storage is SoA: $f[k\,N_xN_y + jN_x + i]$. 72 B/cell DRAM traffic. Tests AoS/SoA layout, push vs pull streaming, and algebraic factorisations of the BGK kernel.

ising. Two-dimensional Ising model, checkerboard Metropolis Monte Carlo on a periodic $N_x \times N_y$ lattice with $\sigma \in \{\pm 1\}$ stored as int8. Per attempt at site (i, j):

$$h = \sigma_{i\pm1,j} + \sigma_{i,j\pm1} \in \{-4, \ldots, 4\}, \quad \Delta E = 2J\sigma\, h, \quad \text{accept} \iff u < \exp(-\beta\,\Delta E). \tag{8}$$

A precomputed five-entry p_{accept} table and a counter-based Murmur-fmix32 PRNG yield bit-exact CPU/GPU agreement: verification is byte-equality on the spin array. 2 B/site/sweep ceiling.

2.4 Irregular memory and atomics (regime R4)

lj. Lennard-Jones molecular dynamics with a cell-list spatial hash. Three kernels per step — `clear_cells`, `build_cells` (atomic scatter), `step` (27-neighbour-cell pair force + integration):

$$\mathbf{F}_i = \sum_{j \in \mathcal{N}(i)} -24\big(2r_{ij}^{-12} - r_{ij}^{-6}\big)\, r_{ij}^{-2}\,(\mathbf{r}_j - \mathbf{r}_i), \qquad r_{ij} < r_{\mathrm{cut}} = 2.5, \tag{9}$$

with minimum-image periodic wrap. Cell occupancy is built via `atomic_fetch_add` on a per-cell counter. Tests load balancing, atomic contention, and the irregular access patterns of neighbour-list MD.

2.5 Multi-kernel reductions (regime R5)

gradshaf. Grad-Shafranov fixed-boundary equilibrium via Picard iteration, two kernels per outer step: an interior max-reduction $\psi_{\mathrm{axis}} = \max_{(i,j)\in\mathrm{int}} \psi$, then a variable-coefficient 5-point stencil with nonlinear source:

$$\psi_{\mathrm{norm}} = \psi/\psi_{\mathrm{axis}}, \quad J = R\,p_{\mathrm{axis}} \cdot 4\psi_{\mathrm{norm}}(1 - \psi_{\mathrm{norm}}) \cdot \mathbf{1}[0 < \psi_{\mathrm{norm}} < 1], \tag{10}$$

$$\Delta^*\psi = a_W\psi_W + a_E\psi_E + a_N\psi_N + a_S\psi_S + a_C\psi_C, \tag{11}$$

$$\psi^{n+1} = \psi^n + \omega\,(-\mu_0 RJ - \Delta^*\psi)/a_C, \tag{12}$$

with R-dependent $a_{W,E} = 1/dR^2 \pm 1/(2RdR)$, $a_{N,S} = 1/dZ^2$, $a_C = -2(1/dR^2 + 1/dZ^2)$. Tests in-kernel reduction strategies (single-TG vs simdgroup-tree) and multi-kernel composition.

2.6 Data-shuffle / butterfly (regime R6)

fft3d. 3D complex-to-complex forward FFT, fp32, on a power-of-two cube of side N. Convention is unnormalised (matches `numpy.fft.fftn`); storage is row-major `float2`$[N][N][N]$. Three named kernels — `fft3d_x`, `fft3d_y`, `fft3d_z` — are dispatched in sequence with two ping-ponged buffers, each kernel performing a length-N 1D FFT per threadgroup of N cooperating threads:

$$Y[k_1,k_2,k_3] \;=\; \sum_{n_1,n_2,n_3=0}^{N-1} X[n_1,n_2,n_3]\, \exp\!\left(-\frac{2\pi i}{N}(k_1n_1 + k_2n_2 + k_3n_3)\right). \tag{13}$$

Sizes $N \in \{32, 64, 128\}$ cover three working-set regimes: 32^3 ($\sim$256 KB) is SLC-resident and compute-bound; 128^3 ($\sim$16 MB) DRAM-binds. The optimisation surface is dominated by data movement, not arithmetic — bit-reversal vs Stockham auto-sort, twiddle-factor caching, mixed-radix (radix-4, radix-8) butterflies for fewer barriers, and `simd_shuffle` / `simd_shuffle_xor` for

Table 1: Uniform 10-iteration claude-opus-4-7 sweep, Apple M1 Pro. "In-distribution" = gmean(achieved/ceiling) over three training sizes; correctness is a hard gate. "Held-out" = fraction-of-ceiling at the unseen size, then ratio. lbm and wave3d use longer pre-existing runs (25 and 15 iters).

	In-distribution			Held-out			
Task	Seed	Best	×	Seed	Best	×	Outcome
saxpy	0.817	1.022	1.25	79%	78%	0.99	saturated
heat2d	0.609	0.609	1.00	100%	101%	1.01	saturated; no candidate beat seed
wave3d	0.533	0.673	1.26	97%	94%	0.97	saturated
ising	0.066	0.075	1.13	12%	11%	0.96	flat
fft3d	0.162	0.167	1.03	39%	42%	1.08	generalises
nbody	0.019	0.055	2.83	1.0%	1.7%	1.63	generalises
gradshaf	0.022	0.041	1.89	3.1%	5.8%	1.89	generalises (identical)
lj	0.003	0.005	1.77	0.25%	0.45%	1.77	generalises (identical)
lbm	0.395	0.576	1.46	83%	53%	**0.64**	**regresses** (TG geom.)
hmc	0.009	0.093	10.6	0.55%	—	**FAIL**	**wrong answer** at d=24

intra-simdgroup butterflies are all on the table. Tests threadgroup-memory bank-conflict avoidance and the per-axis stride asymmetry (stride 1 along x vs strides N and N^2 along y, z). $\sim 5N \log_2 N$ FLOPs per 1D FFT; effective DRAM traffic 96 B/cell across the three axis passes (16 B read + 16 B write per pass). Verification is max-norm against `numpy.fft.fftn` on a fixed seeded Gaussian input, tolerance $10^{-3}+10^{-3}\|Y\|_\infty$.

A bandwidth-bound smoke-test (`saxpy`) is also included to validate the harness.

3 Harness

Compile and dispatch. The harness uses PyObjC's Metal bindings and runtime-compiles `.metal` source via `MTLDevice.newLibraryWithSource`, avoiding the offline `xcrun metal` toolchain; compile errors are returned as structured strings to the LLM. Buffers are allocated in unified memory; numpy views alias buffer contents with a single copy on read-back. All dispatches for a multi-size run share one `MTLCommandBuffer`, and timings come from `GPUEndTime` − `GPUStartTime` (3 warmup, 10 timed, median reported). The chip is detected from `sysctl` and looked up in a per-family table (M1 through M4) for peak FP32 GFLOPS and DRAM bandwidth; all runs here are on Apple M1 Pro (4500 GFLOPS, 200 GB/s).

Scoring. For a candidate passing correctness at every size, $S = (\prod_i f_i)^{1/n}$ with $f_i = \text{achieved}_i/\text{ceiling}_i$. Any tolerance failure forces S=0; this hard gate prevents trading correctness for speed, and the geometric mean across sizes discourages overfitting to one regime.

Evolution loop. A single-population (μ=1+λ=1) loop. Iteration i shows the LLM the previous candidate, the incumbent best, a short history per iteration, and the original task spec. The LLM returns a Metal source block (multi-kernel tasks emit multiple `kernel` functions in one file). Candidates strictly beating the incumbent are promoted; prompts, raw responses, extended-thinking tokens, sources, and per-size JSON results are persisted. A single `call_llm` bridge dispatches Claude via the Agent SDK and Gemini via `google-genai`.

4 Preliminary results

We run a uniform single-model sweep on Apple M1 Pro: claude-opus-4-7 over each of the ten tasks for ten iterations, μ=1+λ=1, no human prompt intervention. Table 1 reports the in-distribution score (gmean $\text{achieved}/\text{ceiling}$ over the three training sizes) and a held-out evaluation, in which the unmodified seed and the incumbent best are run on a single unseen problem size declared by the task spec (Sec. 2).

In-distribution. Speedups span $1.00\times$ to $10.6\times$. The HMC step (Fig. 1) is the high end: introducing a `template <uint D>` worker dispatched on runtime d at iteration 6 moves d=8 from 121

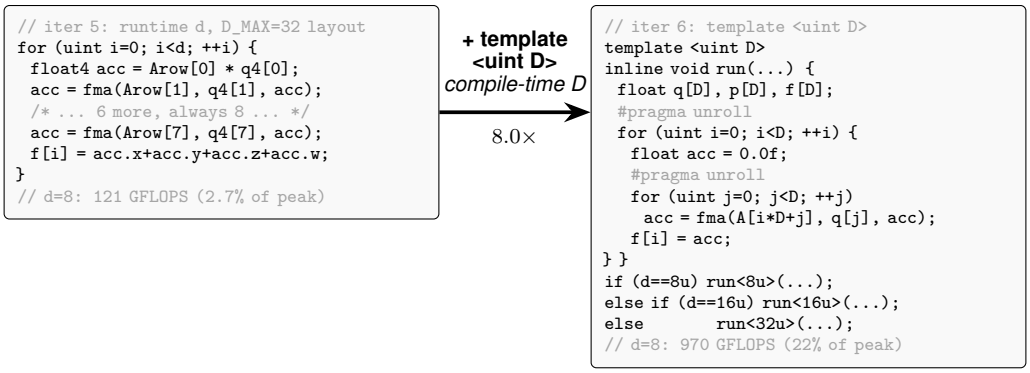

Figure 1: Paradigmatic candidate evolution: HMC, Opus 4.7, iter $5 \to 6$. The structural change is one declaration — `template <uint D>` with three runtime-dispatched instantiations $d \in \{8, 16, 32\}$. Iter 5 had a manually-unrolled float4 inner loop against a fixed $D_{\max}{=}32$ layout (over-computing at $d{=}8$, plus a 4-way horizontal sum); iter 6 sizes `q,p,f` exactly to D so both loops fully unroll into scalar FMAs — moving $d{=}8$ from 121 to 970 GFLOPS in one iteration after five had stalled at 2–4% of peak.

to 970 GFLOPS by enabling full unroll of the inner $A\,q$ matvec. Saturated tasks (saxpy, heat2d, wave3d) sit above 78% of effective DRAM ceiling on the seed; heat2d is the cleanest demonstration that the score is a *hard* signal — zero candidates strictly dominated the seed across all three sizes, so the incumbent stayed at iter 0.

Held-out generalisation is sharper than in-distribution. Three populations separate cleanly. (i) *Generalises*: nbody, fft3d, gradshaf, lj. Gradshaf and lj held-out speedups match their in-distribution speedups to two significant figures (size-invariant simdgroup-tree reduction and cell-list traversal); fft3d 256^3 extrapolates past the largest training size 128^3. (ii) *Saturated*: saxpy, heat2d, wave3d, ising — nothing to win. (iii) *Overfits*, in two qualitatively different ways. **LBM** regresses to $0.64\times$ of the seed on the held-out 192^2 grid: the iter-23 best pins `[[max_total_threads_per_threadgroup(64)]]` with a $32{\times}2$ geometry tuned for the $\{64, 128, 256\}^2$ training sizes; on a non-multiple of 32 the geometry wastes threads at the boundary, and the BGK algebraic fold cannot recover the loss. **HMC** *fails correctness* at $d{=}24$: the template specialisation dispatches `if (d==8) run<8>() ... else run<32>()`, so $d{=}24$ lands in the $D{=}32$ branch; per-thread `q[32], p[32]` and the unrolled matvec process 32 entries against 24-entry data, and sample covariance lands $\sim 10\sigma$ off target. The seed (runtime-d loop) is correct on $d{=}24$ at 0.55% of FP32 peak — a specialisation/generalisation tradeoff the held-out gate exposes and the in-distribution score hides. The LBM 256^2 result above 100% of nominal ceiling reflects working-set residency in the system-level cache: the 8 B/cell roofline is DRAM-only.

Recurring Metal-grammar failures. `[[max_total_threads_per_threadgroup(N)]]` is mis-placed (after the parameter list, or as a standalone statement, instead of on the `kernel void` declaration) on *seven* candidates across five tasks (saxpy, heat2d, nbody, ising, wave3d) — reproduced from earlier opus-4-6 and gemini-3.1-pro runs. `half` is reserved as MSL's fp16 type, breaking `uint half = N >> 1u;` on fft3d iters 1 and 6. C++ lambdas are unsupported, tripping ising iter 2 and wave3d iter 15. Wave3d shows the highest correctness failure rate (10/15): multi-step leapfrog amplifies any sign or indexing error into NaN.

5 Discussion

A roofline-anchored score answers "how close are we to the hardware?" in physical units; the geometric mean across sizes is hard to game by overfitting one regime. Apple Silicon's unified memory halves the host-side scaffolding compared to CUDA, enabling sub-second compile-run-verify cycles per candidate — which matters more than it sounds: an evolutionary loop with tens of candidates needs a fast *harness*, not a fast kernel. Metal's smaller, quirkier surface (simdgroup

intrinsics, threadgroup attributes) also surfaces OOD failures of CUDA-trained LLMs on tasks that are otherwise textbook.

Limitations and future work. The static per-chip ceilings do not account for SLC residency at small sizes (see the LBM 256^2 result above); a workload-aware roofline [4] per task and per size would tighten the score. The single-population $(1+1)$ loop plateaus within 10–25 iterations on most tasks — an island-model or FunSearch-style [1] archive with a novelty signal [2] is the natural next step, particularly for the irregular-memory tasks. Future tasks include sparse linear algebra (SpMV, CG) and cross-chip generalisation (evolve on M2 Pro, evaluate on M3 Max).

References

[1] Romera-Paredes, B. et al. (2024) Mathematical discoveries from program search with large language models. *Nature* **625**, 468–475.

[2] Novikov, A. et al. (2025) AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*.

[3] Ouyang, A. et al. (2025) KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv:2502.10517*.

[4] Williams, S., Waterman, A. & Patterson, D. (2009) Roofline: An insightful visual performance model for multicore architectures. *CACM* **52**(4), 65–76.

[5] Datta, K. et al. (2008) Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures. *Proc. SC '08*.

[6] Nyland, L., Harris, M. & Prins, J. (2007) Fast N-body simulation with CUDA. *GPU Gems 3*, ch. 31.

[7] Schönherr, M. et al. (2011) Multi-thread implementations of the lattice Boltzmann method on non-uniform grids for CPUs and GPUs. *Comput. Math. Appl.* **61**(12), 3730–3743.

[8] Anderson, J. A., Lorenz, C. D. & Travesset, A. (2008) General purpose molecular dynamics simulations fully implemented on graphics processing units. *J. Comput. Phys.* **227**(10), 5342–5359.

[9] Micikevicius, P. (2009) 3D finite difference computation on GPUs using CUDA. *Proc. GPGPU-2*.

[10] Geb, L. et al. (2022) Seamless GPU acceleration for C++ based physics on the Apple M1's unified processing units. *arXiv:2206.01791*.

[11] Govindaraju, N. K. et al. (2008) High performance discrete Fourier transforms on graphics processors. *Proc. SC '08*.

[12] Li, J. et al. (2025) TritonBench: Benchmarking large language model capabilities for generating Triton operators. *Findings of ACL 2025*, pp. 23053–23066.

[13] Saroufim, M. et al. (2025) BackendBench: benchmarking PyTorch-backend kernel generation.

[14] Wen, J. et al. (2025) MultiKernelBench: multi-platform LLM kernel generation across CUDA, Ascend C, and Pallas.

[15] Kalade, S. & Schelle, G. (2025) NPUEval: optimising NPU kernels with LLMs and open-source compilers. *arXiv:2507.14403*.

[16] Nie, J. et al. (2026) KernelCraft: benchmarking agentic close-to-metal kernel generation on emerging hardware. *arXiv:2603.08721*.

[17] Lange, R. T. et al. (2025) Towards robust agentic CUDA kernel benchmarking, verification, and optimisation. *arXiv:2509.14279*.

[18] Guo, P. et al. (2025) EvoEngineer: mastering automated CUDA kernel evolution with large language models. *arXiv:2510.03760*.

A Related work

A.1 LLM-driven kernel-generation benchmarks

A wave of benchmarks has emerged for evaluating LLMs as generators of high-performance GPU kernels. *KernelBench* [3] targets PyTorch ML kernels (GEMM, attention, convolution, normalisation, activation) on CUDA and scores single-shot generation by speedup against the PyTorch eager baseline. *TritonBench* [12] extends to the Triton DSL and adds reference code-similarity to correctness and performance. *BackendBench* [13] evaluates correctness alone for kernels written against the PyTorch backend interface. *MultiKernelBench* [14] broadens the platform set to CUDA, Ascend C, and Pallas while retaining the ML-operator workload set. *NPUEval* [15] targets the AMD AIE NPU in C++ and exposes a tool-use feedback loop with multi-turn regeneration. Most recently, *KernelCraft* [16] benchmarks bare-metal assembly kernel generation on emerging accelerator ISAs (PLENA, AMD NPU, Coral NPU, Sonic BOOM), with native tool interfaces (`check_syntax`, `run_evaluation`, `view_output`, `grep_docs`) and a three-tier ML-task taxonomy (primitive, composite, end-to-end). Across these benchmarks the workload is ML and the headline number is speedup against a vendor or compiler baseline.

A.2 LLM-driven evolutionary code search

The broader paradigm of LLMs as *optimisers* inside evolutionary code-search loops was established by *FunSearch* [1] for symbolic-algorithm discovery and scaled by *AlphaEvolve* [2] across mathematical and algorithmic domains. *AI CUDA*

Table 2: METAL-SCI versus existing LLM kernel-generation benchmarks across the axes that drive the design of our harness. *Domain*: ML = neural-network operators (GEMM, attention, convolution, normalisation, activation); HPC = scientific compute (stencil, N-body, LBM, MD, MCMC, PDE solver, FFT). *Score basis*: what *achieved* is normalised against. *Multi-size*: whether the score requires generalisation across problem sizes. *Search*: the loop structure exposed to the LLM. Within the ML row, all listed benchmarks span a single optimisation regime (matmul-style, with norm/activation epilogues); METAL-SCI spans six structurally distinct regimes (R1–R6 of Section 2).

Benchmark	Target	Domain	Score basis	Multi-size	Search
KernelBench [3]	CUDA	ML	PyTorch speedup	—	single-shot
TritonBench [12]	Triton DSL	ML	speedup + code-sim	—	single-shot
BackendBench [13]	PyTorch backend	ML	correctness only	—	single-shot
MultiKernelBench [14]	CUDA / Ascend C / Pallas	ML	compiler speedup	—	multi-turn
NPUEval [15]	AIE C++ (AMD NPU)	ML	compiler speedup	—	tool-use
KernelCraft [16]	accelerator asm	ML	compiler speedup	cfg. vars	agentic tool-use
METAL-SCI (ours)	**Apple Metal MSL**	**HPC**	**% of roofline**	**3 in + 1 held-out**	**evolutionary** $(1{+}1)$

Engineer [17] and *EvoEngineer* [18] specialise this paradigm to CUDA kernel optimisation, replacing single-shot or agentic regeneration with a population/archive driven by a fitness signal — closer in spirit to our $(\mu{=}1{+}\lambda{=}1)$ loop, but on CUDA ML kernels rather than Apple Silicon scientific kernels.

A.3 Where METAL-SCI differs

Existing kernel-generation benchmarks share three structural choices that METAL-SCI deviates from. *(i) Workload domain.* All listed benchmarks target ML operators, where the optimisation surface is dominated by tiling and warp-level reduction patterns heavily represented in pretraining data. METAL-SCI targets scientific compute — regular and irregular stencils, all-pairs interactions, multi-field Boltzmann updates, neighbour-list molecular dynamics, parallel-chain MCMC, multi-kernel nonlinear PDE solvers, and butterfly spectral transforms — six structurally distinct optimisation regimes (R1–R6 of Section 2) whose canonical recipes [5,6,7,8,11] do not transfer between regimes. *(ii) Score basis.* All listed benchmarks normalise *achieved* against a vendor or compiler baseline (PyTorch eager, compiler-emitted code, expert hand-tuned reference). This couples the headline number to the strength of that baseline. METAL-SCI normalises against the per-chip roofline ceiling (peak FP32 GFLOPS, peak DRAM GB/s), a hardware-anchored target independent of any reference implementation, with a hard correctness gate ($S{=}0$ on tolerance failure) and a geometric mean across three problem sizes that discourages overfit to a single working-set regime. *(iii) Hardware.* No listed benchmark targets Apple Silicon's Metal compute pipeline. Metal is structurally underrepresented in CUDA-centric pretraining data and surfaces a real out-of-distribution generalisation test on Metal-grammar and Metal-stdlib idioms — the `[[max_total_threads_per_threadgroup]]` attribute placement (Sec. 4, seven independent reproductions across five tasks within the opus-4-7 sweep, recurring in earlier opus-4-6 and gemini-3.1-pro runs), `half` as a reserved fp16 keyword, the absence of C++ lambdas, the availability of `simd_shuffle_xor` for intra-simdgroup butterflies, and the absence of `sincospi`. Table 2 summarises these axes against the comparator set.